token on the grid, you can consider all possible responses to all possible open spaces you have, and you can keep doing this until the end of the game. Of course, even for a 3 × 3 board, the first move has nine possible actions, and there are eight possible actions for player 2, and then seven possible actions for player 1 again, so the number of possible trajectories (the game tree) becomes quite large, but a brute force exhaustive search like this could be guaranteed win at Tic-Tac-Toe assuming the opponent isn't using the same approach.

For a game like chess, the game tree is far too large to use a completely brute force search of the game tree; one must necessarily limit the number of potential moves to consider. Deep Blue used a tree-search algorithm that is more efficient than exhaustive search but still involved no learning. It still amounted to searching possible trajectories and just computing which ones led to winning states.

Another approach is the *Monte Carlo tree search*, in which you use some mechanism of randomly sampling a set of potential actions and expanding the tree from there, rather than considering *all* possible actions. The AlphaGo algorithm developed by DeepMind to play the game Go used a deep neural network to evaluate which actions were worth doing a tree search on and also to decide the value of selected moves. Therefore, AlphaGo combined brute force search with deep neural networks to get the best of both. These types of combination algorithms are currently state-of-the-art for games in the class of chess and Go.

11.3. The end

Thank you for reading our book! We really hope you have learned a satisfying amount about deep reinforcement learning. Please reach out to us in the forums at Manning.com with any questions or comments. We look forward to hearing from you.